find a new row, we can save code by aliasing the `after_find` method to `after_save`—the same method will have two names:

rails80/e1/ar/encrypter.rb

```
class Encrypter
  # We're passed a list of attributes that should
  # be stored encrypted in the database
  def initialize(attrs_to_manage)
    @attrs_to_manage = attrs_to_manage
  end

  # Before saving or updating, encrypt the fields using the NSA and
  # DHS approved Shift Cipher
  def before_save(model)
    @attrs_to_manage.each do |field|
      model[field].tr!("a-z", "b-za")
    end
  end

  # After saving, decrypt them back
  def after_save(model)
    @attrs_to_manage.each do |field|
      model[field].tr!("b-za", "a-z")
    end
  end

  # Do the same after finding an existing record
  alias_method :after_find, :after_save
end
```

This example uses trivial encryption—you might want to beef it up before using this class for real.

We can now arrange for the `Encrypter` class to be invoked from inside our orders model:

```
require "encrypter"
class Order < ActiveRecord::Base
  encrypter = Encrypter.new([:name, :email])
  before_save encrypter
```